Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice

David Adrian, Karthikeyan Bhargavan, Zakir Durumeric, Pierrick Gaudry, Matthew Green, J. Alex Halderman, Nadia Heninger, Drew Springall, Emmanuel Thomé, Luke Valenta, Benjamin VanderSloot, Eric Wustrow, Santiago Zanella-Béguelin, Paul Zimmermann
CCS 2015

Iker Martin Canut
Diciembre 2025

Motivación

Motivación

- Diffie–Hellman se usa para lograr **Forward Secrecy**.

Motivación

- Diffie–Hellman se usa para lograr **Forward Secrecy**.
- En la práctica, muchos servidores reutilizan **los mismos parámetros**.

Motivación

- Diffie–Hellman se usa para lograr **Forward Secrecy**.
- En la práctica, muchos servidores reutilizan **los mismos parámetros**.
- Esto permite ataques amortizados a gran escala.

Introducción

Diffie–Hellman y el DLP

Diffie–Hellman y el DLP

- La seguridad de DH depende del **Problema del Logaritmo Discreto**.

Diffie–Hellman y el DLP

- La seguridad de DH depende del **Problema del Logaritmo Discreto**.
- Dado $y \equiv g^x \pmod{p}$, recuperar x debe ser costoso.

Diffie–Hellman y el DLP

- La seguridad de DH depende del **Problema del Logaritmo Discreto**.
- Dado $y \equiv g^x \pmod{p}$, recuperar x debe ser costoso.
- Para primos grandes y únicos, esto es razonable.

Diffie–Hellman y el DLP

- La seguridad de DH depende del **Problema del Logaritmo Discreto**.
- Dado $y \equiv g^x \pmod{p}$, recuperar x debe ser costoso.
- Para primos grandes y únicos, esto es razonable.
- El problema aparece cuando el primo p es **compartido**.

2.1. El Algoritmo *Number Field Sieve* (NFS)

2.1. El Algoritmo *Number Field Sieve* (NFS)

- El DLP mediante NFS se divide en dos fases con propiedades computacionales muy diferentes:

2.1. El Algoritmo *Number Field Sieve* (NFS)

- El DLP mediante NFS se divide en dos fases con propiedades computacionales muy diferentes:
  1. **Etapa de Precomputación (Costo Alto):** Depende **solo del primo** p.

2.1. El Algoritmo *Number Field Sieve* (NFS)

- El DLP mediante NFS se divide en dos fases con propiedades computacionales muy diferentes:
  1. **Etapa de Precomputación (Costo Alto):** Depende **solo del primo** p.
  2. **Etapa de Descenso (Costo Bajo):** Calcula el logaritmo individual x de $y = g^x$ mód p y es la única fase que depende de y (o g).

2.1. El Algoritmo *Number Field Sieve* (NFS)

- El DLP mediante NFS se divide en dos fases con propiedades computacionales muy diferentes:
  1. **Etapa de Precomputación (Costo Alto):** Depende **solo del primo** p.
  2. **Etapa de Descenso (Costo Bajo):** Calcula el logaritmo individual x de $y = g^x$ mód p y es la única fase que depende de y (o g).
- **Implicación Crítica:** Una gran precomputación para un primo p puede amortizarse para calcular **rápidamente** logaritmos discretos arbitrarios en ese grupo, afectando a **todos** los objetivos que comparten p.

Fases de la Precomputación (Dependientes solo de p)

Fases de la Precomputación (Dependientes solo de p)

- **Selección de Polinomio:** Encontrar un polinomio $f(z)$ (grado 5 o 6) para definir el campo numérico $\mathbb{Q}(z)/f(z)$. (Bajo costo, se paralela bien).

Fases de la Precomputación (Dependientes solo de p)

- **Selección de Polinomio:** Encontrar un polinomio $f(z)$ (grado 5 o 6) para definir el campo numérico $\mathbb{Q}(z)/f(z)$. (Bajo costo, se paralela bien).
- **Cribado (Sieving):** Factorear elementos para encontrar **relaciones** cuyos factores primos sean menores que un límite B (B-smooth). (Costo alto, pero se paraleliza muy bien).

Fases de la Precomputación (Dependientes solo de p)

- **Selección de Polinomio:** Encontrar un polinomio $f(z)$ (grado 5 o 6) para definir el campo numérico $\mathbb{Q}(z)/f(z)$. (Bajo costo, se paralela bien).
- **Cribado (Sieving):** Factorear elementos para encontrar **relaciones** cuyos factores primos sean menores que un límite B (B-smooth). (Costo alto, pero se paraleliza muy bien).
- **Álgebra Lineal:** Construir y resolver una matriz grande y dispersa a partir de las relaciones. Esto resulta en una base de datos de logaritmos de elementos pequeños. (Costo alto, paralelización limitada).

Fase del Descenso (Cálculo del Log. Individual)

Fase del Descenso (Cálculo del Log. Individual)

- Escribe $\log(y)$ en términos de los logs de la base de datos precalculada, usando fases de inicialización y cribados hasta que se puede reconstruir el logaritmo.

Fase del Descenso (Cálculo del Log. Individual)

- Escribe $\log(y)$ en términos de los logs de la base de datos precalculada, usando fases de inicialización y cribados hasta que se puede reconstruir el logaritmo.
- El descenso es mucho más económico que la precomputación.

2.3. Parámetros Inseguros y Estándar

- **Generación Inapropiada de Grupos:** Si el orden del grupo tiene muchos factores primos pequeños.

2.3. Parámetros Inseguros y Estándar

- **Generación Inapropiada de Grupos:** Si el orden del grupo tiene muchos factores primos pequeños.
  - Esto se evita típicamente usando primos "seguros" ($p - 1 = 2q$, con q primo).

2.3. Parámetros Inseguros y Estándar

- **Generación Inapropiada de Grupos:** Si el orden del grupo tiene muchos factores primos pequeños.
  - Esto se evita típicamente usando primos "seguros" ($p - 1 = 2q$, con q primo).
  - Pero estos también tienen ataques.

2.3. Parámetros Inseguros y Estándar

- **Generación Inapropiada de Grupos:** Si el orden del grupo tiene muchos factores primos pequeños.
  - Esto se evita típicamente usando primos "seguros" ($p - 1 = 2q$, con q primo).
  - Pero estos también tienen ataques.
- **Primos Estándar:** La **reutilización masiva** de parámetros DH estandarizados (como los **Grupos Oakley** de 768, 1024 y 1536 bits), los cuales son seguros.

2.3. Parámetros Inseguros y Estándar

- **Generación Inapropiada de Grupos:** Si el orden del grupo tiene muchos factores primos pequeños.
  - Esto se evita típicamente usando primos "seguros" ($p - 1 = 2q$, con q primo).
  - Pero estos también tienen ataques.
- **Primos Estándar:** La **reutilización masiva** de parámetros DH estandarizados (como los **Grupos Oakley** de 768, 1024 y 1536 bits), los cuales son seguros.
  - Pero esto convierte ataques costosos en rupturas devastadoras.

Ataques Activos en TLS: Logjam

3. Atacando TLS

3. Atacando TLS

- TLS soporta DH como un método popular de intercambio de claves, y la mayoría de los sitios HTTPS populares lo permiten, con primos de 1024 bits.

3. Atacando TLS

- TLS soporta DH como un método popular de intercambio de claves, y la mayoría de los sitios HTTPS populares lo permiten, con primos de 1024 bits.
- Sin embargo, un número menor de servidores también soporta DH de "grado de exportación"de 512 bits.

3.1. DHE y Criptografía de Exportación

3.1. DHE y Criptografía de Exportación

- **DHE (Diffie-Hellman Efímero):** El servidor elige los parámetros (p, g).

3.1. DHE y Criptografía de Exportación

- **DHE (Diffie-Hellman Efímero):** El servidor elige los parámetros (p, g).
- **DHE_EXPORT (512 bits):** Introducido en SSL 3.0/TLS 1.0 para cumplir con las **restricciones de EE. UU. de los 90**, limitando los primos a 512 bits.

3.1. DHE y Criptografía de Exportación

- **DHE (Diffie-Hellman Efímero):** El servidor elige los parámetros (p, g).
- **DHE_EXPORT (512 bits):** Introducido en SSL 3.0/TLS 1.0 para cumplir con las **restricciones de EE. UU. de los 90**, limitando los primos a 512 bits.
- **Uso en la Práctica:**

3.1. DHE y Criptografía de Exportación

- **DHE (Diffie-Hellman Efímero):** El servidor elige los parámetros (p, g).
- **DHE_EXPORT (512 bits):** Introducido en SSL 3.0/TLS 1.0 para cumplir con las **restricciones de EE. UU. de los 90**, limitando los primos a 512 bits.
- **Uso en la Práctica:**
  - El 8.4 % de los sitios HTTPS del Alexa Top 1M soportaban DHE_EXPORT.

3.1. DHE y Criptografía de Exportación

- **DHE (Diffie-Hellman Efímero):** El servidor elige los parámetros (p, g).
- **DHE_EXPORT (512 bits):** Introducido en SSL 3.0/TLS 1.0 para cumplir con las **restricciones de EE. UU. de los 90**, limitando los primos a 512 bits.
- **Uso en la Práctica:**
  - El 8.4 % de los sitios HTTPS del Alexa Top 1M soportaban DHE_EXPORT.
  - **92.3 % de estos dominios** usaban uno de los dos primos de 512 bits más populares, facilitando el criptoanálisis a gran escala.

3.2. Logjam: Ataque

3.2. Logjam: Ataque

Logjam es un ataque MitM que fuerza una conexión DHE a utilizar un grupo DHE_EXPORT (512 bits), explotando un **fallo del protocolo TLS**:

3.2. Logjam: Ataque

Logjam es un ataque MitM que fuerza una conexión DHE a utilizar un grupo DHE_EXPORT (512 bits), explotando un **fallo del protocolo TLS**: el mensaje ServerKeyExchange no incluye el ciphersuite específico elegido.

3.2. Logjam: Ataque

Logjam es un ataque MitM que fuerza una conexión DHE a utilizar un grupo DHE_EXPORT (512 bits), explotando un **fallo del protocolo TLS**: el mensaje ServerKeyExchange no incluye el ciphersuite específico elegido.

A diferencia de FREAK (error de implementación en RSA), Logjam es una falla inherente del protocolo TLS.

3.2. Logjam: Degradación Activa a DHE_EXPORT

3.2. Logjam: Degradación Activa a DHE_EXPORT

1. **Modificación del Cliente:** El atacante reescribe el `ClientHello` para ofrecer el `DHE_EXPORT` ciphersuite, el cual es aceptado por el servidor.

3.2. Logjam: Degradación Activa a DHE_EXPORT

1. **Modificación del Cliente:** El atacante reescribe el `ClientHello` para ofrecer el `DHE_EXPORT` ciphersuite, el cual es aceptado por el servidor.
2. **Modificación del Servidor:** El servidor envía un `ServerHello` indicando `DHE_EXPORT` y un `ServerKeyExchange` con los parámetros débiles (p_{512}, g, g^b).

3.2. Logjam: Degradación Activa a DHE_EXPORT

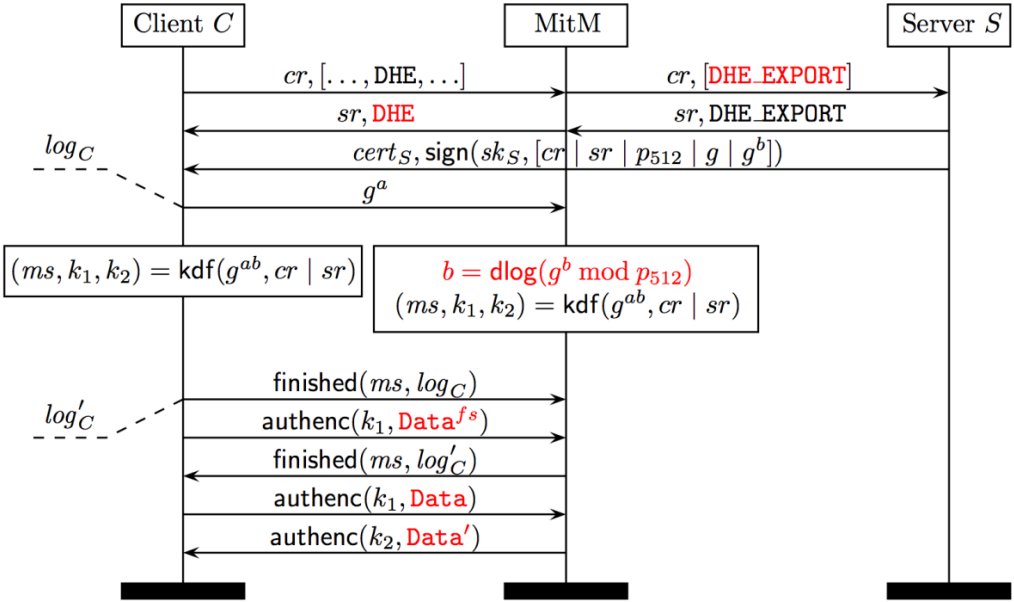
1. **Modificación del Cliente:** El atacante reescribe el `ClientHello` para ofrecer el `DHE_EXPORT` ciphersuite, el cual es aceptado por el servidor.
2. **Modificación del Servidor:** El servidor envía un `ServerHello` indicando `DHE_EXPORT` y un `ServerKeyExchange` con los parámetros débiles (p_{512}, g, g^b).
3. **Fallo Crítico:** El atacante reescribe el `ServerHello` de vuelta al cliente, indicando un ciphersuite DHE **no exportable** que coincide.

3.2. Logjam: Degradación Activa a DHE_EXPORT

1. **Modificación del Cliente:** El atacante reescribe el `ClientHello` para ofrecer el `DHE_EXPORT` ciphersuite, el cual es aceptado por el servidor.
2. **Modificación del Servidor:** El servidor envía un `ServerHello` indicando `DHE_EXPORT` y un `ServerKeyExchange` con los parámetros débiles (p_{512}, g, g^b).
3. **Fallo Crítico:** El atacante reescribe el `ServerHello` de vuelta al cliente, indicando un ciphersuite DHE **no exportable** que coincide.
4. **Engaño al Cliente:** El cliente acepta los parámetros de 512 bits como válidos para DHE estándar, ya que la porción firmada del `ServerKeyExchange` **no incluye el ciphersuite elegido**.

3.2. Logjam: Degradación Activa a DHE_EXPORT

1. **Modificación del Cliente:** El atacante reescribe el `ClientHello` para ofrecer el `DHE_EXPORT` ciphersuite, el cual es aceptado por el servidor.
2. **Modificación del Servidor:** El servidor envía un `ServerHello` indicando `DHE_EXPORT` y un `ServerKeyExchange` con los parámetros débiles (p_{512}, g, g^b).
3. **Fallo Crítico:** El atacante reescribe el `ServerHello` de vuelta al cliente, indicando un ciphersuite DHE **no exportable** que coincide.
4. **Engaño al Cliente:** El cliente acepta los parámetros de 512 bits como válidos para DHE estándar, ya que la porción firmada del `ServerKeyExchange` **no incluye el ciphersuite elegido**.
5. **Toma de Control:** Si el atacante calcula $b = \text{dlog}(g^b \text{ mód } p_{512})$ en tiempo cercano al real, puede **derivar la clave maestra** (g^{ab}).


Client C
MitM
Server S
$cr, [\ldots, \texttt{DHE}, \ldots]$
$cr, [\texttt{DHE_EXPORT}]$
$sr, \texttt{DHE}$
$sr, \texttt{DHE_EXPORT}$
log_C
$cert_S, \mathsf{sign}(sk_S, [cr \mid sr \mid p_{512} \mid g \mid g^b])$
g^a
$(ms, k_1, k_2) = \mathsf{kdf}(g^{ab}, cr \mid sr)$
$b = \mathsf{dlog}(g^b \bmod p_{512})$
$(ms, k_1, k_2) = \mathsf{kdf}(g^{ab}, cr \mid sr)$
$\mathsf{finished}(ms, log_C)$
log'_C
$\mathsf{authenc}(k_1, \texttt{Data}^{fs})$
$\mathsf{finished}(ms, log'_C)$
$\mathsf{authenc}(k_1, \texttt{Data})$
$\mathsf{authenc}(k_2, \texttt{Data}')$

3.3 Computación de Logaritmos Discretos de 512 bits

3.3 Computación de Logaritmos Discretos de 512 bits

Se utilizó **CADO-NFS** modificado para los primos DHE_EXPORT.

3.3 Computación de Logaritmos Discretos de 512 bits

Se utilizó **CADO-NFS** modificado para los primos DHE_EXPORT.

- **Precomputación:** Duró **7 días** por primo.

3.3 Computación de Logaritmos Discretos de 512 bits

Se utilizó **CADO-NFS** modificado para los primos DHE_EXPORT.

- **Precomputación:** Duró **7 días** por primo.
  - Cribado: 15 horas ($\sim 21,400$ core-hours).
  - Álgebra Lineal: 120 horas ($\sim 60,000$ core-hours).

3.3 Computación de Logaritmos Discretos de 512 bits

Se utilizó **CADO-NFS** modificado para los primos DHE_EXPORT.

- **Precomputación:** Duró **7 días** por primo.
  - Cribado: 15 horas ($\sim 21,400$ core-hours).
  - Álgebra Lineal: 120 horas ($\sim 60,000$ core-hours).
- **Descenso Individual:** El cálculo de un logaritmo discreto individual tuvo una **mediana de 70 segundos**, variando entre 34 y 206 segundos.

3.4. Ataque Activo

Para falsificar el mensaje `Finished` y tomar el control, el atacante debe mitigar el retraso de ~ 70 segundos.

3.4. Ataque Activo

Para falsificar el mensaje `Finished` y tomar el control, el atacante debe mitigar el retraso de ~ 70 segundos.

- **Alertas de Advertencia:** Mantener la conexión viva enviando **alertas de advertencia TLS** (ej. Firefox).

3.4. Ataque Activo

Para falsificar el mensaje `Finished` y tomar el control, el atacante debe mitigar el retraso de ~ 70 segundos.

- **Alertas de Advertencia:** Mantener la conexión viva enviando **alertas de advertencia TLS** (ej. Firefox).
- **Caché de Claves Efímeras:** Explotar servidores que reutilizan el valor g^b (17 % de hosts DHE), permitiendo que el logaritmo se calcule **offline**.

3.4. Ataque Activo

Para falsificar el mensaje `Finished` y tomar el control, el atacante debe mitigar el retraso de ~ 70 segundos.

- **Alertas de Advertencia:** Mantener la conexión viva enviando **alertas de advertencia TLS** (ej. Firefox).
- **Caché de Claves Efímeras:** Explotar servidores que reutilizan el valor g^b (17 % de hosts DHE), permitiendo que el logaritmo se calcule **offline**.
- **TLS False Start:** Descifrar datos de aplicación sensibles (ej. contraseñas) que el cliente envía **antes** de recibir el mensaje `Finished` del servidor.

Amenazas a Nivel Estatal (768 y 1024 bits)

4.1. Escalabilidad y Viabilidad del Ataque NFS

4.1. Escalabilidad y Viabilidad del Ataque NFS

Tabla 1: Estimación de Costos de Criptoanálisis DH (Basado en)

Tamaño	Precomputación (Total)	Descenso	Viabilidad

Tabla 1: Estimación de Costos de Criptoanálisis DH (Basado en)

Tamaño	Precomputación (Total)	Descenso	Viabilidad
512 bits	89,000 core-hours	70 segundos	**Académica / Moderada**

4.1. Escalabilidad y Viabilidad del Ataque NFS

Tabla 1: Estimación de Costos de Criptoanálisis DH (Basado en)

Tamaño	Precomputación (Total)	Descenso	Viabilidad
512 bits	89,000 core-hours	70 segundos	**Académica / Moderada**
768 bits	36,500 core-years	2 core-days	**Académica Avanzada**

4.1. Escalabilidad y Viabilidad del Ataque NFS

Tabla 1: Estimación de Costos de Criptoanálisis DH (Basado en)

Tamaño	Precomputación (Total)	Descenso	Viabilidad
512 bits	89,000 core-hours	70 segundos	**Académica / Moderada**
768 bits	36,500 core-years	2 core-days	**Académica Avanzada**
1024 bits	45 Millones de core-years	30 core-days	**Nación-Estado**

4.1. Escalabilidad y Viabilidad del Ataque NFS

Tabla 1: Estimación de Costos de Criptoanálisis DH (Basado en)

Tamaño	Precomputación (Total)	Descenso	Viabilidad
512 bits	89,000 core-hours	70 segundos	**Académica / Moderada**
768 bits	36,500 core-years	2 core-days	**Académica Avanzada**
1024 bits	45 Millones de core-years	30 core-days	**Nación-Estado**

- Aunque parezca mucho, está al alcance.

4.1. Escalabilidad y Viabilidad del Ataque NFS

Tabla 1: Estimación de Costos de Criptoanálisis DH (Basado en)

Tamaño	Precomputación (Total)	Descenso	Viabilidad
512 bits	89,000 core-hours	70 segundos	**Académica / Moderada**
768 bits	36,500 core-years	2 core-days	**Académica Avanzada**
1024 bits	45 Millones de core-years	30 core-days	**Nación-Estado**

- Aunque parezca mucho, está al alcance.
- Hay lugar para mejores métodos y optimizaciones.

4.1. Escalabilidad y Viabilidad del Ataque NFS

Tabla 1: Estimación de Costos de Criptoanálisis DH (Basado en)

Tamaño	Precomputación (Total)	Descenso	Viabilidad
512 bits	89,000 core-hours	70 segundos	**Académica / Moderada**
768 bits	36,500 core-years	2 core-days	**Académica Avanzada**
1024 bits	45 Millones de core-years	30 core-days	**Nación-Estado**

- Aunque parezca mucho, está al alcance.
- Hay lugar para mejores métodos y optimizaciones.
- Hay agencias con recursos para esto, por ejemplo

4.1. Escalabilidad y Viabilidad del Ataque NFS

Tabla 1: Estimación de Costos de Criptoanálisis DH (Basado en)

Tamaño	Precomputación (Total)	Descenso	Viabilidad
512 bits	89,000 core-hours	70 segundos	**Académica / Moderada**
768 bits	36,500 core-years	2 core-days	**Académica Avanzada**
1024 bits	45 Millones de core-years	30 core-days	**Nación-Estado**

- Aunque parezca mucho, está al alcance.
- Hay lugar para mejores métodos y optimizaciones.
- Hay agencias con recursos para esto, por ejemplo NSA.

4.3. Impacto de la Ruptura DH-1024

4.3. Impacto de la Ruptura DH-1024

Una ruptura exitosa de los grupos DH de 1024 bits más comunes tendría un impacto masivo debido a la reutilización de parámetros.

- **HTTPS (TLS):**

4.3. Impacto de la Ruptura DH-1024

Una ruptura exitosa de los grupos DH de 1024 bits más comunes tendría un impacto masivo debido a la reutilización de parámetros.

- **HTTPS (TLS):**
  - Aproximadamente el **24.0 %** de las conexiones de navegador con los sitios Top 1M negocian DHE con uno de los diez primos de 1024 bits más populares.

4.3. Impacto de la Ruptura DH-1024

Una ruptura exitosa de los grupos DH de 1024 bits más comunes tendría un impacto masivo debido a la reutilización de parámetros.

- **HTTPS (TLS):**
  - Aproximadamente el **24.0 %** de las conexiones de navegador con los sitios Top 1M negocian DHE con uno de los diez primos de 1024 bits más populares.
  - La precomputación de **un solo primo de 1024 bits** permitiría la escucha pasiva del **17.9 %** de las conexiones Top 1M.
- **VPN (IKE):**

4.3. Impacto de la Ruptura DH-1024

Una ruptura exitosa de los grupos DH de 1024 bits más comunes tendría un impacto masivo debido a la reutilización de parámetros.

- **HTTPS (TLS):**
  - Aproximadamente el **24.0 %** de las conexiones de navegador con los sitios Top 1M negocian DHE con uno de los diez primos de 1024 bits más populares.
  - La precomputación de **un solo primo de 1024 bits** permitiría la escucha pasiva del **17.9 %** de las conexiones Top 1M.
- **VPN (IKE):**
  - El **66.1 % de los servidores IKEv1** y el **63.9 % de los servidores IKEv2** perfilados preferían el Grupo Oakley 2 de 1024 bits.

4.3. Impacto de la Ruptura DH-1024

Una ruptura exitosa de los grupos DH de 1024 bits más comunes tendría un impacto masivo debido a la reutilización de parámetros.

- **HTTPS (TLS):**
  - Aproximadamente el **24.0 %** de las conexiones de navegador con los sitios Top 1M negocian DHE con uno de los diez primos de 1024 bits más populares.
  - La precomputación de **un solo primo de 1024 bits** permitiría la escucha pasiva del **17.9 %** de las conexiones Top 1M.
- **VPN (IKE):**
  - El **66.1 % de los servidores IKEv1** y el **63.9 % de los servidores IKEv2** perfilados preferían el Grupo Oakley 2 de 1024 bits.
- **SSH:**

4.3. Impacto de la Ruptura DH-1024

Una ruptura exitosa de los grupos DH de 1024 bits más comunes tendría un impacto masivo debido a la reutilización de parámetros.

- **HTTPS (TLS):**
  - Aproximadamente el **24.0 %** de las conexiones de navegador con los sitios Top 1M negocian DHE con uno de los diez primos de 1024 bits más populares.
  - La precomputación de **un solo primo de 1024 bits** permitiría la escucha pasiva del **17.9 %** de las conexiones Top 1M.
- **VPN (IKE):**
  - El **66.1 % de los servidores IKEv1** y el **63.9 % de los servidores IKEv2** perfilados preferían el Grupo Oakley 2 de 1024 bits.
- **SSH:**
  - Un atacante que precomputara el Grupo Oakley 2 (1024 bits) podría escuchar pasivamente las conexiones al **25.7 % de los servidores SSH** accesibles.

Recomendaciones y Mitigaciones

5. Recomendaciones Estratégicas

- 1. Transición a Curvas Elípticas (ECDH):

5. Recomendaciones Estratégicas

- **1. Transición a Curvas Elípticas (ECDH):**
  - Es la solución a largo plazo más efectiva, ya que evita todos los ataques criptoanalíticos factibles conocidos contra DH mod p.

5. Recomendaciones Estratégicas

- **1. Transición a Curvas Elípticas (ECDH):**
  - Es la solución a largo plazo más efectiva, ya que evita todos los ataques criptoanalíticos factibles conocidos contra DH mod p.
  - NSA?

5. Recomendaciones Estratégicas

- **1. Transición a Curvas Elípticas (ECDH):**
  - Es la solución a largo plazo más efectiva, ya que evita todos los ataques criptoanalíticos factibles conocidos contra DH mod p.
  - NSA?
- **2. Aumentar la Fuerza Mínima de la Clave:**

5. Recomendaciones Estratégicas

- **1. Transición a Curvas Elípticas (ECDH):**
  - Es la solución a largo plazo más efectiva, ya que evita todos los ataques criptoanalíticos factibles conocidos contra DH mod p.
  - NSA?
- **2. Aumentar la Fuerza Mínima de la Clave:**
  - Los servidores deben **deshabilitar DHE_EXPORT** y configurar DHE para usar primos de **2048 bits o más**.

5. Recomendaciones Estratégicas

- **1. Transición a Curvas Elípticas (ECDH):**
  - Es la solución a largo plazo más efectiva, ya que evita todos los ataques criptoanalíticos factibles conocidos contra DH mod p.
  - NSA?
- **2. Aumentar la Fuerza Mínima de la Clave:**
  - Los servidores deben **deshabilitar DHE_EXPORT** y configurar DHE para usar primos de **2048 bits o más**.
  - La precomputación para 2048 bits es $\sim 10^9$ veces más difícil que para 1024 bits.

5. Recomendaciones Estratégicas

- **1. Transición a Curvas Elípticas (ECDH):**
  - Es la solución a largo plazo más efectiva, ya que evita todos los ataques criptoanalíticos factibles conocidos contra DH mod p.
  - NSA?
- **2. Aumentar la Fuerza Mínima de la Clave:**
  - Los servidores deben **deshabilitar DHE_EXPORT** y configurar DHE para usar primos de **2048 bits o más**.
  - La precomputación para 2048 bits es $\sim 10^9$ veces más difícil que para 1024 bits.
- **3. Evitar Primos Fijos de 1024 bits:**

5. Recomendaciones Estratégicas

- **1. Transición a Curvas Elípticas (ECDH):**
  - Es la solución a largo plazo más efectiva, ya que evita todos los ataques criptoanalíticos factibles conocidos contra DH mod p.
  - NSA?
- **2. Aumentar la Fuerza Mínima de la Clave:**
  - Los servidores deben **deshabilitar DHE_EXPORT** y configurar DHE para usar primos de **2048 bits o más**.
  - La precomputación para 2048 bits es $\sim 10^9$ veces más difícil que para 1024 bits.
- **3. Evitar Primos Fijos de 1024 bits:**
  - Las implementaciones que requieran 1024 bits deben **generar grupos nuevos** para mitigar el daño de la precomputación de NFS sobre grupos fijos comunes.

5. Recomendaciones Estratégicas

- **1. Transición a Curvas Elípticas (ECDH):**
  - Es la solución a largo plazo más efectiva, ya que evita todos los ataques criptoanalíticos factibles conocidos contra DH mod p.
  - NSA?
- **2. Aumentar la Fuerza Mínima de la Clave:**
  - Los servidores deben **deshabilitar DHE_EXPORT** y configurar DHE para usar primos de **2048 bits o más**.
  - La precomputación para 2048 bits es $\sim 10^9$ veces más difícil que para 1024 bits.
- **3. Evitar Primos Fijos de 1024 bits:**
  - Las implementaciones que requieran 1024 bits deben **generar grupos nuevos** para mitigar el daño de la precomputación de NFS sobre grupos fijos comunes.
- **4. No Debilitar la Criptografía:**

5. Recomendaciones Estratégicas

- **1. Transición a Curvas Elípticas (ECDH):**
  - Es la solución a largo plazo más efectiva, ya que evita todos los ataques criptoanalíticos factibles conocidos contra DH mod p.
  - NSA?
- **2. Aumentar la Fuerza Mínima de la Clave:**
  - Los servidores deben **deshabilitar DHE_EXPORT** y configurar DHE para usar primos de **2048 bits o más**.
  - La precomputación para 2048 bits es $\sim 10^9$ veces más difícil que para 1024 bits.
- **3. Evitar Primos Fijos de 1024 bits:**
  - Las implementaciones que requieran 1024 bits deben **generar grupos nuevos** para mitigar el daño de la precomputación de NFS sobre grupos fijos comunes.
- **4. No Debilitar la Criptografía:**
  - Los ataques DHE_EXPORT (Logjam y FREAK) demuestran los **efectos de debilitar deliberadamente la criptografía**.

Conclusión

7. Conclusión: La Falla de la Reutilización de Primos

7. Conclusión: La Falla de la Reutilización de Primos

- **Fallo Operacional:** DH es vulnerable a la estrategia del *number field sieve*, donde una única precomputación costosa reduce drásticamente el costo de los ataques posteriores.

7. Conclusión: La Falla de la Reutilización de Primos

- **Fallo Operacional:** DH es vulnerable a la estrategia del *number field sieve*, donde una única precomputación costosa reduce drásticamente el costo de los ataques posteriores.
- Este hecho fue conocido por los criptógrafos, pero **no fue ampliamente comprendido por los constructores de sistemas**.

7. Conclusión: La Falla de la Reutilización de Primos

- **Fallo Operacional:** DH es vulnerable a la estrategia del *number field sieve*, donde una única precomputación costosa reduce drásticamente el costo de los ataques posteriores.
- Este hecho fue conocido por los criptógrafos, pero **no fue ampliamente comprendido por los constructores de sistemas**.
- **Impacto No Anticipado:** La seguridad de una gran parte de Internet dependería de la **reutilización de unos pocos primos**.